

BÀI 8. XÂY DỰNG CHỨC NĂNG CRUD & UPLOAD FILE

Bài trước, chúng ta đã có một CSDL hoạt động với dữ liệu mẫu. Nhưng làm thế nào để thêm một sản phẩm mới, sửa giá, hay xóa một mặt hàng đã hết kinh doanh? Chắc chắn không thể vào CSDL sửa bằng tay được. Bài này, chúng ta sẽ xây dựng một khu vực quản trị chuyên nghiệp với đầy đủ các chức năng **CRUD (Tạo, Đọc, Cập nhật, Xóa)**, và tích hợp một trong những tính năng thú vị nhất: upload hình ảnh sản phẩm.

MỤC TIÊU BÀI HỌC

Sau khi hoàn thành Bài học này, bạn sẽ có khả năng:

- **Thiết kế và triển khai** một luồng CRUD hoàn chỉnh trong ASP.NET Core.
- **Giải thích rành mạch** vai trò của các cặp Action [HttpGet] và [HttpPost] trong việc xử lý form.
- **Làm chủ kỹ thuật Model Binding** để ASP.NET Core tự động "điền form" vào các đối tượng C# cho bạn.
- **Tự tay code chức năng upload file** một cách an toàn, lưu file lên server và lưu đường dẫn vào CSDL.

NỘI DUNG

8.1. Xây dựng chức năng CRUD (Create, Read, Update, Delete)

CRUD là viết tắt của bốn hoạt động cơ bản nhất mà bạn có thể thực hiện trên dữ liệu. Trong ASP.NET Core MVC, mỗi hoạt động này thường tương ứng với một hoặc nhiều Action trong Controller.

8.1.1. Read (Đọc/Xem)

- **Mục tiêu:** Hiển thị một hoặc nhiều bản ghi.
- **Triển khai:** Thường là một Action [HttpGet] duy nhất.
 - **Index():** Lấy **tất cả** bản ghi từ repository và truyền chúng đến một View để hiển thị dưới dạng bảng.

8.1.2. Create (Tạo mới)

- **Mục tiêu:** Tạo một bản ghi mới.
- **Triển khai:** Luôn là một **cặp Action** có cùng tên nhưng khác phương thức HTTP.
 1. Create() [**HttpGet**]: Người dùng bấm vào nút "Thêm mới". Action này được gọi. Nhiệm vụ của nó rất đơn giản: **hiển thị một form trống** để người dùng nhập liệu. Nó trả về View(new Product());.
 2. Create(Product product) [**HttpPost**]: Người dùng điền thông tin và nhấn "Lưu". Dữ liệu form được gửi đi. Action này được gọi. Nhiệm vụ của nó là: **nhận dữ liệu, xử lý (validation), lưu vào CSDL**, rồi chuyển hướng người dùng về trang danh sách.

8.1.3. Update (Cập nhật)

- **Mục tiêu:** Chỉnh sửa một bản ghi đã tồn tại.
- **Triển khai:** Tương tự Create, đây cũng là một **cặp Action**.
 1. Edit(int id) [**HttpGet**]: Người dùng bấm vào nút "Sửa" của một sản phẩm cụ thể. Action này nhận id của sản phẩm đó. Nhiệm vụ: lấy sản phẩm từ CSDL dựa trên id, sau đó hiển thị một form đã được điền sẵn thông tin cũ để người dùng sửa.
 2. Edit(Product product) [**HttpPost**]: Người dùng sửa xong và nhấn "Lưu". Action này nhận đối tượng product đã được cập nhật, xử lý và lưu thay đổi vào CSDL, rồi chuyển hướng về trang danh sách.

8.1.4. Delete (Xóa)

- **Mục tiêu:** Xóa một bản ghi.
- **Triển khai:** Thường là một Action [**HttpPost**] duy nhất.
 - Delete(int productID) [**HttpPost**]: Tại sao là POST mà không phải GET? Đây là một quy ước bảo mật quan trọng. Nếu dùng GET (một link <a> đơn giản), các công cụ tìm kiếm có thể vô tình "bò" vào và xóa hết dữ liệu của bạn! Dùng POST (bên trong một form) đảm bảo hành động xóa phải xuất phát từ một ý định rõ ràng của người dùng (nhấn nút submit).

8.2. Model Binding - "Người trợ lý" tự động điền form

Bạn có để ý trong các Action [HttpPost] ở trên, chúng ta nhận vào một đối tượng Product đã được điền sẵn dữ liệu? Đó là nhờ vào Model Binding.

8.2.1. Model Binding là gì?

Hãy tưởng tượng bạn có một tờ đơn đăng ký (form HTML) và một hồ sơ giấy (đối tượng C#). Thay vì phải tự tay đọc từng dòng trên tờ đơn rồi chép vào hồ sơ, bạn có một "người trợ lý" siêu thông minh tự động làm việc đó. Đó chính là **Model Binding**.

Model Binding là cơ chế tự động của ASP.NET Core, nó đọc dữ liệu từ một HTTP request và "ánh xạ" (map) chúng vào các tham số của một Action method, thường là các đối tượng phức tạp (model).

8.2.2. Nó hoạt động như thế nào?

Chìa khóa nằm ở thuộc tính name của các thẻ input trong form HTML. Khi submit một form, trình duyệt sẽ đóng gói dữ liệu dưới dạng các cặp key=value.

- Ví dụ: Name=Bóng%20đá&Description=Mô%20tả&Price=25.00

Model Binder sẽ nhìn vào các key này (Name, Description, Price) và tìm các thuộc tính có tên **trùng khớp** trong class model mà Action yêu cầu (ví dụ class Product). Nó sẽ tự động gán các value tương ứng vào các thuộc đó.

Ví dụ minh họa cụ thể:

- Model C# của bạn:

```
//C#  
public class Product {  
    public string Name { get; set; }  
    public decimal Price { get; set; }  
}
```

- Form HTML của bạn:

```
<form action="/Product/Create" method="post">  
    <input type="text" name="Name" />  
    <input type="text" name="Price" />  
    <button type="submit">Save</button>  
</form>
```

– Action trong Controller:

```
//C#  
[HttpPost]  
public IActionResult Create(Product product) {  
    // Đến đây, product.Name và product.Price đã được tự động điền giá trị.  
    // ...  
}
```

8.2.3. Model Binding Lấy Dữ Liệu Từ Đâu?

Ngoài dữ liệu từ form (form values), người "trợ lý" Model Binder này rất thông minh và sẽ tự động tìm kiếm dữ liệu ở nhiều nơi khác trong một HTTP request theo một thứ tự ưu tiên cụ thể.

Thứ tự mặc định là:

1. **Form Values:** Đây là nguồn được ưu tiên hàng đầu. Dữ liệu được gửi đi trong body của một request, thường là từ một thẻ <form> với phương thức POST.
 - Ví dụ: Name=Bóng đá&Price=25.00
2. **Route Data:** Dữ liệu từ các tham số được định nghĩa trong URL pattern.
 - Ví dụ: Trong URL /Product/Details/5, nếu route của bạn là {controller}/{action}/{id}, Model Binder sẽ tìm thấy giá trị id = 5.
3. **Query String:** Dữ liệu nằm ở phần đuôi của URL, sau dấu ?.
 - Ví dụ: Trong URL /Product/Search?category=Dụng cụ thể thao, Model Binder sẽ tìm thấy category = "Dụng cụ thể thao".

Quá trình hoạt động: Khi cần điền giá trị cho một tham số (ví dụ string category), Model Binder sẽ:

- Đầu tiên, nó tìm trong **Form Values** xem có key nào tên là category không. Nếu có, nó sẽ lấy giá trị đó và dừng lại.
- Nếu không tìm thấy, nó tiếp tục tìm trong **Route Data**.
- Nếu vẫn không thấy, nó sẽ tìm trong **Query String**.
- Nếu không tìm thấy ở bất cứ đâu, tham số sẽ nhận giá trị mặc định của kiểu dữ liệu đó (ví dụ null cho string, 0 cho int).

8.2.4. Chuyện Gì Xảy Ra Khi Dữ Liệu Không Khớp?

Đây là một điểm rất quan trọng, và cách Model Binder xử lý nó rất tinh tế.

– Trường hợp 1: Tên không khớp

- Nếu form có `<input name="productName">` nhưng model chỉ có thuộc tính `Name`.
- **Kết quả:** Model Binder sẽ **bỏ qua** dữ liệu đó. Nó sẽ không cố gắng "đoán". Thuộc tính `Name` trong đối tượng `C#` của bạn sẽ không được gán giá trị và sẽ giữ giá trị mặc định của nó (null đối với kiểu `string`).

• Ví dụ:

○ **Request data:** `productName=Bóng đá`

○ **Model class:** `public class Product { public string Name { get; set; } }`

- **Kết quả binding:** Đối tượng `Product` được tạo ra nhưng `product.Name` sẽ là null. Sẽ **không có lỗi** nào xảy ra, chỉ đơn giản là dữ liệu không được ánh xạ.

– Trường hợp 2: Kiểu dữ liệu không hợp lệ

- Người dùng nhập chữ "miễn phí" vào ô `Price` (thuộc tính `Price` trong model là kiểu `decimal`).

- **Kết quả:** Đây là lúc cơ chế validation của ASP.NET Core vào cuộc.

1. Model Binder sẽ cố gắng chuyển đổi chuỗi "miễn phí" thành kiểu `decimal`. Việc này chắc chắn sẽ **thất bại**.

2. Khi chuyển đổi thất bại, Model Binder sẽ **không** gây ra exception làm sập ứng dụng. Thay vào đó, nó làm hai việc:

- Gán giá trị mặc định cho thuộc tính đó. `product.Price` sẽ được gán giá trị 0.
- Thêm một lỗi vào một đối tượng đặc biệt gọi là **ModelState**. Đối tượng này theo dõi tất cả các lỗi xảy ra trong quá trình binding và validation.

Trong Controller, chúng ta có thể (và nên) kiểm tra `ModelState` trước khi xử lý:

```
//C#  
[HttpPost]  
public IActionResult Create(Product product)  
{
```

```
// ModelState.IsValid sẽ trả về 'false' nếu có lỗi binding
if (ModelState.IsValid)
{
    // Nếu không có lỗi, tiến hành lưu vào database
    _repository.SaveProduct(product);
    return RedirectToAction("Index");
}
else
{
    // Nếu có lỗi, trả lại View cho người dùng sửa lại.
    // Các Tag Helper như 'asp-validation-for' sẽ tự động hiển thị lỗi.
    return View(product);
}
}
```

Tóm lại: Model Binding là một cơ chế mạnh mẽ và linh hoạt. Nó không chỉ tự động điền dữ liệu từ nhiều nguồn mà còn tích hợp sẵn với hệ thống validation, giúp bạn xử lý các trường hợp người dùng nhập liệu sai một cách an toàn và chuyên nghiệp.

8.2.5. Áp dụng vào SportsStore

Xét code cho view Edit.cshtml tương ứng với Action Edit(Product product, IFormFile? image) mà chúng ta đã phân tích.

Code này sử dụng các **Tag Helper** của ASP.NET Core (asp-for, asp-action, asp-validation-summary...) để tạo form một cách mạnh mẽ và giảm thiểu lỗi.

– Code cho file Views/Admin/Edit.cshtml

```
@* Khai báo model mà View này sẽ làm việc cùng *@
@model Product
@{
    ViewData["Title"] = "Edit Product";
    Layout = "_AdminLayout";
}
<h3 class="mb-3">Edit Product: @Model.Name</h3>
<form asp-action="Edit" method="post" enctype="multipart/form-data">
    <div asp-validation-summary="All" class="text-danger"></div>
    <input type="hidden" asp-for="ProductID" />
    <div class="form-group mb-3">
        <label asp-for="Name"></label>
        <input asp-for="Name" class="form-control" />
    </div>
</form>
```

```

        <span asp-validation-for="Name" class="text-danger"></span>
    </div>
    <div class="form-group mb-3">
        <label asp-for="Description"></label>
        <textarea asp-for="Description" class="form-control"
rows="3"></textarea>
        <span asp-validation-for="Description" class="text-danger"></span>
    </div>
    <div class="form-group mb-3">
        <label asp-for="Category"></label>
        <input asp-for="Category" class="form-control" />
        <span asp-validation-for="Category" class="text-danger"></span>
    </div>
    <div class="form-group mb-3">
        <label asp-for="Price"></label>
        <input asp-for="Price" class="form-control" />
        <span asp-validation-for="Price" class="text-danger"></span>
    </div>
    <div class="form-group mb-3">
        <label>Image</label>
        <input type="file" name="image" class="form-control" />
    </div>
    <div class="text-center">
        <button class="btn btn-primary" type="submit">Save</button>
        <a asp-action="Index" class="btn btn-secondary">Cancel</a>
    </div>
</form>

```

– Giải thích các điểm chính

- **@model Product:** Dòng này khai báo rằng View được "gõ mạnh" (strongly-typed) với model Product. Điều này cho phép bạn sử dụng asp-for để liên kết trực tiếp các thẻ input với thuộc tính của model.
- **enctype="multipart/form-data":** Đây là thuộc tính **quan trọng nhất** cho việc upload file. Nếu thiếu nó, file sẽ không được gửi đi và tham số IFormFile? image trong Action của bạn sẽ luôn là null.
- **<input type="hidden" asp-for="ProductID" />:** Khi form được submit, chúng ta cần cho server biết ID của sản phẩm đang được chỉnh sửa. Trường ẩn này sẽ gửi giá trị ProductID đi cùng với các dữ liệu khác. Model Binder sẽ tự động

gán nó vào `product.ProductID`.

- **asp-for="PropertyName"**: Tag Helper này rất thông minh. Nó tự động:
 - Tạo id và name cho thẻ input khớp với tên thuộc tính (ví dụ: `name="Name"`).
 - Hiển thị giá trị hiện tại của thuộc tính trong ô input khi trang được tải.
 - Giúp hiển thị thông báo lỗi validation.
- `<input type="file" name="image" />`: Đây là điểm mấu chốt.
 - Chúng ta không dùng `asp-for` ở đây vì class `Product` không có thuộc tính `Image` kiểu `IFormFile` (trong kịch bản này).
 - Thuộc tính `name="image"` được đặt thủ công. Model Binder sẽ dựa vào cái tên "image" này để tìm và gán file được upload vào tham số `IFormFile? image` trong Action `Edit`.
- **asp-validation-for** và **asp-validation-summary**: Các Tag Helper này sẽ tự động hiển thị các thông báo lỗi nếu `ModelState` trong Controller không hợp lệ, giúp người dùng biết họ cần sửa thông tin gì.

– Phân tích một Action [`HttpPost`]:

Model Binding là cơ chế tự động của ASP.NET Core giúp chuyển đổi dữ liệu từ HTTP request thành các đối tượng C# một cách thông minh. Hãy phân tích cách nó hoạt động với Action [`HttpPost`] trong `SportsStore`.

- Xét Action `Edit` sau:

```
//C#

[HttpPost]
public async Task<IActionResult> Edit(Product product, IFormFile? image)
{
    // ... code xử lý logic ...
}
```

Khi người dùng điền thông tin vào form và nhấn "Lưu", một chuỗi sự kiện tự động diễn ra.

– Quá trình hoạt động của Model Binding

1./Request được gửi đi

Trình duyệt đóng gói tất cả dữ liệu từ form (tên, giá, mô tả, và file ảnh) vào một HTTP POST request. Vì có file đính kèm, request này sẽ có thuộc tính `enctype="multipart/form-data"`.

2./Routing xác định Action

ASP.NET Core nhận request và dựa vào URL (ví dụ: `/Admin/Edit/12`) cùng với phương thức (POST), nó xác định phải gọi đến Action Edit trong AdminController.

3./Model Binder được kích hoạt

Đây là lúc "phép màu" xảy ra. Trước khi chạy code bên trong Action Edit, hệ thống thấy rằng Action này cần hai tham số: một đối tượng Product và một đối tượng IFormFile. Model Binder sẽ tự động chuẩn bị sẵn các tham số này.

Nó xử lý tuần tự từng tham số một:

1. Xử lý tham số Product product

- **Tạo đối tượng rỗng:** Model Binder tạo một instance mới của class Product: `new Product()`.
- **Quét và Ánh xạ:** Nó bắt đầu quét dữ liệu trong request (ưu tiên từ form) để tìm các cặp `key=value` có **key** trùng với tên các thuộc tính của class Product.
 - Thấy `key="ProductID"`, nó gán `product.ProductID = 12`.
 - Thấy `key="Name"`, nó gán `product.Name = "Giá trị người dùng nhập"`.
 - Thấy `key="Price"`, nó lấy chuỗi giá trị, tự động chuyển đổi sang kiểu decimal và gán vào `product.Price`.
 - Nó tiếp tục làm tương tự cho Description, Category,...
 - **Quan trọng:** Khi thấy `key="image"` (từ thẻ `<input type="file" name="image">`), nó sẽ **bỏ qua** vì trong class Product (theo kịch bản này) không có thuộc tính nào tên là Image.

Kết thúc quá trình này, Model Binder đã có một đối tượng product chứa đầy đủ dữ liệu từ các trường input.

2. Xử lý tham số IFormFile? image

- **Tìm kiếm theo tên:** Sau khi xong với product, Model Binder chuyển sang

tham số image. Nó lại quét request, lần này tìm một key có tên chính xác là image.

- **Đóng gói file:** Nó tìm thấy dữ liệu file được gửi lên, sau đó lấy toàn bộ thông tin (tên, loại, kích thước, nội dung) và gói gọn vào một đối tượng IFormFile.
- **Gán giá trị:** Đối tượng IFormFile này được gán cho tham số image.
- **Trường hợp không có file:** Nếu người dùng không chọn file nào, request sẽ không chứa key="image". Vì tham số được định nghĩa là IFormFile? (cho phép null), giá trị của image lúc này sẽ là null và không có lỗi nào xảy ra.

Kết quả cuối cùng

Khi chương trình bắt đầu thực thi code bên trong Action Edit, bạn đã có sẵn:

- **product:** Một đối tượng Product đã được điền đầy đủ thông tin.
- **image:** Một đối tượng IFormFile chứa file ảnh người dùng đã chọn, hoặc null.

Nhờ Model Binding, bạn có thể làm việc trực tiếp với các đối tượng C# tường minh và an toàn, **hoàn toàn không cần phải viết code thủ công và nhàm chán** như Request.Form["Name"] nữa.

8.3. Xử lý File Upload - Gửi "bưu kiện" qua Web

8.3.1. Phía Client: Chuẩn bị "thùng hàng" đặc biệt

Một form HTML mặc định chỉ có thể gửi được text. Để gửi được file (hình ảnh, video, pdf...), bạn phải "báo" cho trình duyệt rằng đây là một lô hàng đặc biệt, chứa nhiều loại dữ liệu khác nhau.

– `<form enctype="multipart/form-data">`:

- enctype là viết tắt của "encoding type".
- multipart/form-data có nghĩa là dữ liệu của form sẽ được chia thành **nhiều phần (multipart)**, một phần cho dữ liệu text (tên, giá...), và các phần khác cho mỗi file được đính kèm.

– `<input type="file">`:

- Tạo ra một nút "Choose File" cho phép người dùng chọn một file từ máy tính của họ.

8.3.2. Phía Server: Nhận "bưu kiện" với IFormFile

ASP.NET Core đơn giản hóa việc nhận file bằng một interface cực kỳ tiện lợi: IFormFile.

Khi form được submit, Model Binder sẽ thấy có một phần dữ liệu là file, nó sẽ đóng gói toàn bộ thông tin của file đó vào một đối tượng IFormFile và truyền vào Action cho bạn.

Đối tượng IFormFile chứa mọi thứ bạn cần:

- **FileName:** Tên file gốc từ máy client.
- **Length:** Kích thước file (tính bằng byte).
- **ContentType:** Loại file (ví dụ image/jpeg).
- **OpenReadStream():** Phương thức quan trọng nhất, cho phép bạn đọc nội dung của file dưới dạng một Stream.

8.3.3. Các bước xử lý file trên Server (Áp dụng vào SportsStore)

Quá trình này có thể được chia thành 4 bước chính:

1. **Chuẩn bị Form phía Client (View):** Cho phép người dùng chọn và gửi file.
2. **Cập nhật Model và ViewModel:** Để có thể "hứng" được file được gửi lên.
3. **Xử lý lưu file ở phía Server (Controller):** Nhận file, kiểm tra và lưu vào một thư mục trên server.
4. **Hiển thị hình ảnh:** Lấy đường dẫn đã lưu và hiển thị ra cho người dùng.

8.3.4. Bước 1: Chuẩn bị Form ở phía Client (View)

Để trình duyệt có thể gửi file lên server, bạn cần chỉnh sửa file View (.cshtml), thường là file Edit.cshtml hoặc Create.cshtml.

- **Thêm enctype="multipart/form-data" vào thẻ <form>:** Đây là thuộc tính **bắt buộc**. Nó báo cho trình duyệt biết rằng form này không chỉ gửi dữ liệu dạng text thông thường mà còn chứa cả file. Nếu không có thuộc tính này, server sẽ không nhận được file.

```
<form asp-action="Edit" method="post" enctype="multipart/form-data">  
    </form>
```

- **Thêm thẻ <input type="file">**: Đây là thành phần cho phép người dùng bấm vào và chọn một file từ máy tính của họ.

```
//HTML
<div class="form-group">
    <label>Image</label>
    <input type="file" name="Image" class="form-control" />
</div>
```

Lưu ý: Thuộc tính name="Image" rất quan trọng. Model Binder sẽ dựa vào tên này để ánh xạ file được upload vào thuộc tính tương ứng trong Model của bạn ở bước sau.

8.3.5. Bước 2: Cập nhật Model (hoặc ViewModel)

Để "húng" được file mà người dùng gửi lên, bạn cần thêm một thuộc tính kiểu IFormFile vào class Model hoặc ViewModel của mình. Trong ngữ cảnh SportsStore, chúng ta có thể thêm nó vào class Product.

- IFormFile là một interface đặc biệt trong ASP.NET Core, đại diện cho file được gửi lên cùng với HTTP request. Nó chứa mọi thông tin về file như:
 - FileName: Tên file gốc (ví dụ: bong-da-dep.jpg).
 - Length: Kích thước file (tính bằng byte).
 - ContentType: Loại file (ví dụ: image/jpeg).
 - OpenReadStream(): Một phương thức để đọc nội dung file dưới dạng một Stream.

```
//C#
using Microsoft.AspNetCore.Http;
using System.ComponentModel.DataAnnotations.Schema; // Cần thêm để sử dụng [NotMapped]
public class Product
{
    public long? ProductID { get; set; }
    public string Name { get; set; }
    public string Description { get; set; }
    public decimal Price { get; set; }
    public string Category { get; set; }
    // Thêm thuộc tính này để lưu đường dẫn ảnh trong CSDL
    public string? ImageUrl { get; set; }
}
```

```
// Thuộc tính này chỉ dùng để nhận file từ request, không lưu vào CSDL
[NotMapped]
public IFormFile Image { get; set; }
}
```

– Quan trọng:

- Chúng ta thêm thuộc tính ImageUrl (kiểu string) để lưu đường dẫn tới file ảnh trong cơ sở dữ liệu. Chúng ta không bao giờ lưu trực tiếp file ảnh vào database vì nó rất tốn tài nguyên và làm chậm hệ thống.
- Thuộc tính Image (kiểu IFormFile) được đánh dấu bằng [NotMapped]. Điều này báo cho Entity Framework Core biết rằng không cần tạo một cột tên là Image trong bảng Products của database. Nó chỉ là một thuộc tính "trung chuyển" để nhận file.

8.3.6. Bước 3: Xử lý lưu file ở phía Server (Controller)

Đây là bước cốt lõi, diễn ra trong Action method của AdminController (thường là Edit hoặc Create).

```
//C#
[HttpPost]
public async Task<IActionResult> Edit(Product product)
// Model Binding đã tự động điền dữ liệu vào 'product', bao gồm cả file ảnh
{
    if (ModelState.IsValid)
    {
        // 1. Kiểm tra xem người dùng có upload file mới không
        if (product.Image != null && product.Image.Length > 0)
        {
            // 2. Tạo đường dẫn tuyệt đối để lưu file
            // Lấy đường dẫn đến thư mục gốc của web (wwwroot)
            var wwwRootPath = _webHostEnvironment.WebRootPath;
            // Tạo một tên file duy nhất để tránh trùng lặp (ví dụ: sử dụng GUID)
            var fileName = Guid.NewGuid().ToString() +
            Path.GetExtension(product.Image.FileName);
            // Kết hợp đường dẫn thư mục wwwroot, thư mục con 'images', và tên file mới
            var filePath = Path.Combine(wwwRootPath, "images", fileName);
            // 3. Lưu file vào thư mục trên server
```

```
        using (var stream = new FileStream(filePath, FileMode.Create))
        {
            await product.Image.CopyToAsync(stream);
        }

        // 4. Cập nhật thuộc tính ImageUrl SP với đường dẫn mới
        // Chỉ lưu đường dẫn tương đối để hiển thị trên web
        product.ImageUrl = "/images/" + fileName;
    }

    // 5. Lưu thông tin sản phẩm (gồm ImageUrl mới) vào database
    _repository.SaveProduct(product);
    TempData["message"] = $"{product.Name} has been saved";
    return RedirectToAction("Index");
}

else
{
    // Có lỗi validation, trả về view với dữ liệu đã nhập
    return View(product);
}
}
```

– Giải thích chi tiết các bước trong Controller:

1. **Kiểm tra file:** Luôn kiểm tra xem product.Image có khác null không và Length (kích thước) có lớn hơn 0 không. Điều này đảm bảo người dùng thực sự đã chọn một file để upload.
2. **Tạo đường dẫn lưu trữ:**
 - Chúng ta lưu file vào thư mục wwwroot/images. Thư mục wwwroot là nơi chứa các file tĩnh (CSS, JS, hình ảnh) có thể được truy cập trực tiếp từ trình duyệt.
 - **Tạo tên file duy nhất:** Rất quan trọng! Nếu hai người dùng cùng upload file có tên product.jpg, file sau sẽ ghi đè file trước. Sử dụng Guid.NewGuid() là cách phổ biến để tạo một chuỗi ngẫu nhiên không bao giờ trùng lặp. Chúng ta giữ lại phần đuôi file gốc (.jpg, .png).
3. **Lưu file:**
 - FileStream được dùng để tạo một file mới trên đĩa tại filePath.
 - product.Image.CopyToAsync(stream) là lệnh đọc toàn bộ nội dung của file

được upload và chép nó vào FileStream vừa tạo. Quá trình này hoàn tất việc lưu file vật lý lên server.

4. **Cập nhật Model:** Sau khi lưu file thành công, chúng ta gán đường dẫn tương đối (ví dụ: /images/ten-file-duy-nhat.jpg) vào thuộc tính product.ImageUrl.
5. **Lưu vào Database:** Cuối cùng, gọi phương thức `_repository.SaveProduct(product)`. Entity Framework sẽ cập nhật tất cả các thuộc tính của sản phẩm, bao gồm cả ImageUrl mới vào cơ sở dữ liệu.

***Quan trọng:** Không bao giờ lưu đường dẫn vật lý đầy đủ (D:\project\wwwroot\images\ball.jpg) vào CSDL. Hãy lưu đường dẫn tương đối so với wwwroot (ví dụ: /images/ball.jpg).*

***Tại sao?** Vì khi bạn triển khai lên một server khác, đường dẫn vật lý sẽ thay đổi, nhưng đường dẫn tương đối thì không. Điều này giúp ứng dụng của bạn có thể di động (portable).*

8.3.7. Bước 4: Hiển thị hình ảnh

Bây giờ bạn đã có đường dẫn ảnh lưu trong thuộc tính ImageUrl của mỗi sản phẩm. Để hiển thị nó ra, bạn chỉ cần dùng thẻ `<img>` trong các file View (Index.cshtml hoặc file hiển thị sản phẩm cho khách hàng).

```
//HTML
@model IEnumerable<Product>
//...
@foreach (var p in Model) {
    <tr>
        <td>
            @if (!string.IsNullOrEmpty(p.ImageUrl))
            {
                
            }
        </td>
        <td>@p.Name</td>
        <td>@p.Price.ToString("c")</td>
        ...
    </tr>
}
```

Thẻ sẽ lấy đường dẫn từ p.ImageUrl (ví dụ: /images/ten-file-duy-nhat.jpg) và trình duyệt sẽ tự động gửi yêu cầu đến server để tải và hiển thị đúng hình ảnh đó.

8.4. Hướng dẫn thực hành

8.4.1.1. Tạo AdminController và các Action

- **Thao tác:** Trong Controllers, tạo một Controller rỗng tên AdminController.cs.
- Code Edit (POST) được phân tích:

```
//C#
[HttpPost]
public async Task<IActionResult> Edit(Product product, IFormFile?
imageFile)
{
    // ModelState.IsValid sẽ được học ở Bài 9, tạm thời coi như nó luôn
    đúng
    if (ModelState.IsValid)
    {
        // BƯỚC 1: KIỂM TRA FILE
        // Kiểm tra xem người dùng có thực sự chọn một file để upload
        không.
        if (imageFile != null && imageFile.Length > 0)
        {
            // BƯỚC 2: TẠO ĐƯỜNG DẪN LƯU FILE
            // Lấy tên thư mục gốc của ứng dụng web (ví dụ:
            C:\...\SportsStore.WebUI)
            var webRootPath = Directory.GetCurrentDirectory();
            // Lấy tên file gốc một cách an toàn
            var fileName = Path.GetFileName(imageFile.FileName);
            // Kết hợp các thành phần để tạo ra đường dẫn vật lý đầy đủ và
            an toàn.
            // Ví dụ: C:\...\SportsStore.WebUI\wwwroot\images\ball.jpg
            var filePath = Path.Combine(webRootPath, "wwwroot/images",
            fileName);

            // BƯỚC 3: LƯU FILE
            // Mở một luồng (stream) để ghi vào file tại đường dẫn đã tạo.
            // 'using' đảm bảo stream sẽ được đóng lại đúng cách ngay cả
            khi có lỗi xảy ra.
            using (var stream = new FileStream(filePath, FileMode.Create))
            {
```

```
        // Sao chép nội dung của file được upload (imageFile) vào
        luồng của file đích.

        // Đây là quá trình ghi file thực sự. Dùng bản Async để
        không block thread chính.

        await imageFile.CopyToAsync(stream);

    }

    // BƯỚC 4: LƯU ĐƯỜNG DẪN TƯƠNG ĐỐI VÀO CSDL

    // Cập nhật thuộc tính của đối tượng product để lưu đường dẫn
    web, không phải đường dẫn vật lý.

    product.ImageUrl = "/images/" + fileName;

    }

    // Gọi repository để lưu đối tượng product (đã được cập nhật
    ImageUrl) vào CSDL.

    _repository.SaveProduct(product);

    // Dùng TempData để hiển thị một thông báo thành công ở request
    tiếp theo.

    TempData["message"] = $"{product.Name} has been saved";

    // Chuyển hướng người dùng về trang danh sách sản phẩm.

    return RedirectToAction("Index");

    }

    else

    {

        // Nếu model không hợp lệ, hiển thị lại form edit với dữ liệu người
        dùng đã nhập.

        return View(product);

    }

    }

}
```

CÂU HỎI LÝ THUYẾT

1. Tại sao các Action Create và Edit cần cả phiên bản [HttpGet] và [HttpPost]? Mỗi phiên bản có nhiệm vụ gì?
2. Thuộc tính HTML nào trong thẻ <input> quyết định tên của thuộc tính trong Model C# mà nó sẽ được binding tới?
3. Tại sao chúng ta phải thêm enctype="multipart/form-data" vào thẻ <form> khi cần upload file?
4. Khi lưu thông tin ảnh vào CSDL, tại sao chúng ta nên lưu đường dẫn tương đối (/images/tenfile.jpg) thay vì đường dẫn vật lý đầy đủ?

TỔNG KẾT BÀI HỌC

Bài này, chúng ta đã xây dựng nên "trung tâm điều hành" cho cửa hàng. Bạn đã học được:

- **Luồng CRUD chuẩn** trong MVC, một kỹ năng nền tảng cho bất kỳ ứng dụng nào cần quản lý dữ liệu.
- **Sức mạnh của Model Binding**, giúp code của bạn gọn gàng và ít lỗi hơn.
- **Quy trình xử lý file upload hoàn chỉnh**, từ client đến server, một kỹ năng cực kỳ thực tế và được yêu cầu cao.

Khu vực quản trị của bạn giờ đây đã hoạt động đầy đủ, cho phép bạn thực sự quản lý cửa hàng của mình.

BÀI TẬP TỰ LÀM

1. **Hoàn thiện Action Delete:** Dựa trên logic đã học, hãy viết code hoàn chỉnh cho Action [HttpPost] Delete(int productID).

Nâng cao: Sau khi xóa sản phẩm khỏi CSDL, hãy viết thêm code để tìm và xóa cả file ảnh tương ứng của nó trên server để tránh lưu trữ file rác.

2. **Thêm trường Manufacturer:** Áp dụng kiến thức từ các Bài trước và Bài này, hãy thực hiện một chu trình hoàn chỉnh:
 - Thêm thuộc tính Manufacturer (nhà sản xuất) vào class Product.
 - Tạo và áp dụng một migration mới để cập nhật CSDL.
 - Cập nhật View Edit.cshtml để thêm một ô input cho Manufacturer.
 - Chạy ứng dụng và kiểm tra xem bạn có thể thêm/sửa thông tin nhà sản xuất cho sản phẩm hay không.

TÀI LIỆU THAM KHẢO

- [1]. **Model Binding in ASP.NET Core:** [Microsoft Docs - Model Binding](#)
- [2]. **File uploads in ASP.NET Core:** [Microsoft Docs - File Uploads](#)
- [3]. **The form Tag Helper:** [Microsoft Docs - Form Tag Helper](#)